Análise dos Resultados — Sprint 5

Construção de um LLM do Zero · Pedro Devens Dellalibera e João Gabriel Trevisol

Esta análise interpreta os experimentos registrados no documento de experimentação da Sprint 5, relacionando a perda, o treinamento e as estratégias de geração aos conceitos do Capítulo 5 e às sprints anteriores. Todos os números vêm da execução em GPU (RTX 3050, PyTorch 2.15.0+cu134) registrada naquele documento.

1. Por que o modelo começa com perda ≈ 11

A cross-entropy é a média de −ln(p) sobre os tokens corretos. Um modelo que distribui a probabilidade de forma uniforme entre os 50.257 tokens do vocabulário dá p = 1/50.257 a cada alvo, e −ln(1/50.257) = ln(50.257) ≈ **10,82**. A perda inicial medida (**10,9876** no treino e **10,9832** na validação) fica um pouco acima desse valor porque os pesos aleatórios não produzem uma distribuição exatamente uniforme: eles favorecem alguns tokens ao acaso, e errar com confiança custa mais do que errar com incerteza.

Essa é a referência para ler todo o resto: qualquer perda abaixo de 10,8 significa que o modelo aprendeu algo sobre o texto. A perplexidade traduz isso para uma escala intuitiva: cerca de **59.135** tokens possíveis no início — praticamente o vocabulário inteiro — contra **1,62** no fim do treino. Ou seja, depois de 10 épocas o modelo hesita entre menos de dois tokens a cada posição do texto de treino.

Vale registrar que o cálculo manual (softmax → probabilidade do alvo → logaritmo → média → troca de sinal) e a `cross_entropy` do PyTorch deram 10,793963 e 10,793964. A diferença na sexta casa decimal é só arredondamento de ponto flutuante, e confirma que a função do PyTorch faz exatamente esses passos internamente.

2. O modelo aprende, mas aprende o texto e não a língua

A perda de treino cai de forma contínua durante as 10 épocas, de 7,17 para **0,4823**, o que mostra que o laço `zero_grad` → `backward` → `step` está funcionando: a cada passo o AdamW move os pesos na direção que aumenta a probabilidade dos tokens corretos. A perda de validação, porém, para de cair na **época 6**, em **6,1247**, e a distância entre as duas curvas chega a **5,8991** no fim.

A explicação está na proporção entre parâmetros e dados. O modelo tem 162,4 milhões de parâmetros e vê apenas 4.608 tokens de treino, cerca de **35 mil parâmetros por token**. Com essa capacidade, o caminho mais fácil para reduzir a perda é decorar a sequência exata do conto.

O texto gerado confirma isso de forma categórica: na época 10, em modo greedy, o modelo reproduz **30 de 30 tokens** literalmente do The Verdict. Não é um trecho parecido — é o conto transcrito. A validação, que vem do final do conto e nunca é vista no treino, não se beneficia dessa memorização.

A comparação com o livro valida a implementação: Raschka reporta 0,569 no treino e 6,373 na validação na mesma configuração, e obtivemos 0,482 e 6,381. A diferença de menos de 0,1 vem da GPU fazer as operações de ponto flutuante em ordem diferente da CPU.

3. Número de épocas: onde parar

O treino de 25 épocas mostra com clareza o ponto em que treinar mais deixa de ajudar: a menor perda de validação aparece na **época 6 (6,1247)**. Dali em diante a perda de treino continua caindo até **0,012** na época 25 — praticamente zero — enquanto a de validação sobe de forma contínua até **7,296**, pior do que o valor que ela tinha na época 2.

Cada época extra custa cerca de **5,3 segundos** e, a partir da época 6, compra apenas memorização. O sinal mais forte disso não está nos números, e sim no texto: nas épocas 10, 15, 20 e 25 a saída em modo greedy é **exatamente a mesma**, caractere por caractere. Depois da época 10 o modelo não aprende nada de novo sobre o conto; apenas aumenta a confiança no que já decorou.

Em um treino real, a época 6 seria o ponto de aplicar early stopping, guardando o checkpoint com a menor perda de validação — algo que o mecanismo de salvamento do `5,4.py` já tornaria possível.

4. Taxa de aprendizado: o passo grande não acelera, desestabiliza

Este experimento produziu o resultado menos intuitivo da sprint. A expectativa seria que taxas maiores convergissem mais rápido, chegando antes a uma perda de treino baixa. Foi o que aconteceu até 0,0005, e o oposto a partir daí.

Com **0,0001** o modelo converge devagar: ao fim de 10 épocas a perda de treino ainda está em 2,869, longe dos 0,482 do treino base. Mas é exatamente por não ter tido tempo de memorizar que ele termina com a melhor perda de validação **final** (6,252).

Com **0,0005**, vizinho dos 0,0004 do livro, o comportamento espelha o treino base: converge até 0,582 e atinge a **melhor validação absoluta de todo o experimento, 6,206 na época 7**.

Com **0,001** e **0,005**, as perdas de treino terminam mais **altas** (4,248 e 5,607), não mais baixas. O passo é grande demais e o otimizador salta por cima do mínimo em vez de descer até ele. O texto gerado torna isso visível sem precisar de gráfico: as duas configurações colapsam em sequências de quebras de linha, ou seja, o modelo desiste de prever conteúdo e passa a repetir o token mais frequente do corpus.

O tempo é praticamente idêntico nas quatro (0,9 min, de 859 a 866 tokens/s), porque a taxa de aprendizado não muda a quantidade de operações — só o tamanho de cada atualização.

5. Dropout: o resultado contraria a teoria, e isso tem explicação

O esperado seria que mais dropout aumentasse a perda de treino (por desligar ativações durante o aprendizado) e diminuísse a de validação (por regularizar). **Aconteceu o oposto nos dois eixos.** A perda de treino caiu conforme o dropout subiu (0,595 → 0,482 → 0,442) e a de validação piorou levemente (6,372 → 6,381 → 6,459). A diferença entre as duas, que é a medida direta do sobreajuste, **aumentou** de 5,778 para 6,017.

A primeira leitura é de cautela metodológica: a diferença entre os extremos na validação é de apenas 0,087, medida sobre um único lote de 512 tokens e com uma única semente por configuração. Uma variação dessa ordem está dentro do que a própria aleatoriedade da inicialização e da ordem dos lotes pode produzir, então não dá para afirmar que o dropout piorou o modelo.

A segunda leitura é a que importa, e é a mesma independentemente do ruído: **com 9 lotes por época e 4.608 tokens de treino, nenhuma taxa de dropout resolve o problema de fundo.** O dropout é uma técnica para impedir que a rede dependa demais de neurônios específicos — ele regulariza, mas não cria informação. Se o conjunto de treino é pequeno demais para representar a língua, o modelo vai

memorizá-lo de um jeito ou de outro, e mexer no `drop_rate` apenas muda o caminho até a memorização.

Vale notar também que o dropout só atua em modo `train()`. Todas as perdas relatadas são medidas com o modelo em `eval()`, portanto sem dropout ativo — o que estamos comparando é o efeito que a regularização teve durante o aprendizado, não durante a medição.

6. Custo computacional

Uma época tem 9 passos e processa 4.608 tokens. O treino base levou **52,23 segundos** para as 10 épocas, ou **882 tokens por segundo**, em uma GPU RTX 3050 de 6 GB. Os oito scripts da sprint, incluindo os 8 treinos dos experimentos, rodaram em **12 minutos e 23 segundos** no total.

O custo cresce linearmente com o número de épocas — 10 épocas em 52 s, 25 épocas em 132 s — e a taxa de aprendizado e o dropout praticamente não o alteram (de 859 a 882 tokens/s entre todas as configurações), porque não mudam o número de operações. Parte do tempo, aliás, não é treino: a avaliação a cada 5 passos e a geração de 50 tokens no fim de cada época também fazem passagens pelo modelo.

Em memória, os pesos ocupam **622,6 MB** em float32 e o checkpoint com o otimizador **1.861,9 MB** — cerca de três vezes mais, porque o AdamW guarda duas médias móveis por parâmetro. Os arquivos por isso ficam fora do Git, como no repositório do livro, e o README explica como gerá-los de novo.

Uma observação sobre o hardware: a mesma carga em CPU levaria da ordem de duas a três horas. A GPU não muda nenhuma conclusão do experimento, mas foi o que tornou viável rodar as oito configurações em uma única sessão.

7. Estratégia de geração × pesos do modelo

O enunciado pede para separar o efeito das estratégias de geração do efeito do treinamento, e a comparação entre o modelo sem treino e o treinado faz exatamente isso. A estratégia só decide **como escolher** entre as probabilidades que o modelo produz; ela não cria conhecimento.

No modelo **sem treino**, a distribuição do próximo token é quase plana: entropia de **10,66** com T = 1, contra um máximo teórico de ln(50.257) ≈ 10,82. Qualquer estratégia gera tokens soltos, sem relação entre si. Os 100% de tokens únicos que aparecem na tabela não indicam boa diversidade — indicam que todos os tokens são igualmente improváveis. Mudar a temperatura ou o top-k muda quais tokens aparecem, nunca o fato de serem aleatórios.

No modelo **treinado**, a distribuição é concentrada: entropia de **3,96** com T = 1, e o token mais provável (`?"`) sozinho carrega 41,8% da massa. O greedy escolhe sempre o mesmo caminho e reproduz o conto — **30 tokens copiados de 30**. A amostragem com T = 1 já produz textos diferentes a cada semente e derruba a cópia para 8,7 tokens; T = 1,5 leva a 1,3, com 100% de tokens únicos, mas à custa da coerência.

O **top-k** corta a cauda da distribuição em vez de achatá-la. Com k = 1 a geração volta a ser determinística, produzindo exatamente o mesmo texto do greedy nas três sementes — a confirmação prática do exercício 5.3. Com k entre 5 e 50 a diversidade aumenta sem os tokens absurdos que a temperatura alta sozinha deixa passar, e a combinação do livro (k = 25, T = 1,4) fica em 6,3 tokens copiados: o melhor equilíbrio entre não repetir o conto e não gerar ruído.

Em uma frase: **coerência vem dos pesos; diversidade e repetição são controladas pela estratégia.**

8. Pesos salvos e reprodutibilidade

Salvar o `state_dict` e carregá-lo em um modelo novo reproduziu os **173 tensores** idênticos, a mesma perda de validação (6,381409 nos dois casos) e o mesmo texto gerado caractere por caractere. O teste tem um controle embutido: antes do `load_state_dict`, o modelo novo — criado com outra semente — tinha pesos diferentes, o que garante que a igualdade posterior vem do carregamento e não de coincidência.

Para isso funcionar, o modelo precisa ser criado com a mesma configuração usada no treino; por isso gravamos o `gpt_config` e o `config_treino` dentro do checkpoint, o que permite identificar o modelo apenas a partir do arquivo, sem depender de anotações externas.

A retomada por mais uma época mostrou a perda de treino caindo de 0,4823 para 0,2919 e a de validação subindo de 6,3814 para 6,5274 — coerente com o sobreajuste já observado no item 2. O contador interno do AdamW foi de 90 para 99 passos, confirmando que o otimizador continuou de onde parou em vez de reiniciar as médias móveis.

9. O que os pesos da OpenAI mostram

O mesmo `GPTModel`, agora com os pesos do GPT-2 124M, obteve perda de validação de **3,5566** no The Verdict, contra **6,3814** do nosso modelo — sem nunca ter visto esse texto. Em perplexidade: o GPT-2 hesita entre cerca de **35** tokens a cada posição, o nosso entre **591**.

A inversão em relação ao treino é o ponto central. Nosso modelo tem perda de **treino** muito menor (0,48 contra 3,75): ele conhece o The Verdict melhor que o GPT-2, porque o decorou. Mas essa vantagem não sobrevive ao primeiro trecho que ele não viu.

A arquitetura é idêntica — o mesmo código da Sprint 4 carrega os dois conjuntos de pesos. A diferença é o volume de dados do pré-treinamento: o GPT-2 foi treinado em cerca de 40 GB de texto da internet, contra os 20 KB nossos. Isso confirma a conclusão do item 2: com tão pouco texto, o nosso modelo aprende o conto e não a língua.

O texto gerado resume tudo sem precisar de métrica. O GPT-2 produz *"Every effort moves you forward. The first step is to understand the importance of your work."* — uma frase original, gramatical e coerente com o contexto. O nosso produz uma transcrição literal do conto.

Limitações desta etapa

- **Dados:** um único conto de 5.146 tokens. Serve para entender o processo, mas não para treinar um modelo que generalize.
- **Validação:** um único lote de 512 tokens, o que torna a perda de validação uma estimativa ruidosa. Pequenas diferenças entre experimentos — como os 0,087 que separam as três taxas de dropout — não devem ser super-interpretadas.
- **Uma semente por configuração:** os experimentos de treino não foram repetidos com outras sementes, então não medimos a variação entre execuções. Esta é a limitação que mais pesa na leitura do item 5.
- **Um parâmetro por vez:** não testamos interações (por exemplo, taxa de aprendizado alta com dropout alto), que o grid completo do livro cobriria.

- **Sem agendamento da taxa de aprendizado:** o livro mostra warmup e cosine decay no apêndice D, que não foram usados aqui. Dado o comportamento instável das taxas 0,001 e 0,005, um agendamento provavelmente mudaria aquele resultado.
- **Métricas de texto simples:** tokens únicos, bigramas repetidos e trecho copiado são indicadores aproximados de diversidade, repetição e memorização, não medidas de qualidade do texto.

O que esta sprint entrega é o ciclo completo de pré-treinamento funcionando sobre a arquitetura das sprints anteriores: avaliar, treinar, gerar, salvar e carregar. A Sprint 6 parte desses pesos (ou dos pesos da OpenAI) para o fine-tuning.